

HW13

Tguo67

Question(s):

Read the related materials, and answer the following questions:

1. In LMCache, how does it mitigate the overhead caused by KV Cache transfer?
2. What are the motivations and insights that support the design of sharing KV Cache in DroidSpeak?

Related materials:

LMCache: <https://arxiv.org/pdf/2510.09665> [Links to an external site.](#)

DroidSpeak: <https://arxiv.org/pdf/2411.02820> [Links to an external site.](#)

Answers:

1. How LMCache mitigates KV-cache transfer overhead

LMCache's main problem: KV pages are small (e.g., 16 tokens per page, 20–63 KB), so naive per-page transfer is bandwidth-inefficient and also interferes with compute. It tackles this with several systems tricks:

(a) Batch and coarsen transfers

- **Configurable chunk size:** Instead of moving per-page KV (tiny), LMCache groups many pages across layers into large “chunks” (default ~256 tokens) using an intermediate GPU buffer, so each GPU↔CPU/SSD/remote transfer is big enough to saturate bandwidth.
- **Batched multi-tier ops:** It batches store/load operations across tiers (GPU→CPU and GPU→SSD, etc.) so different links can be used in parallel rather than sequentially under-utilizing them.
- **Delayed decode storing:** During decoding, it aggregates multiple newly generated pages before offloading, avoiding many small writes.

(b) Overlap KV transfer with compute

- **Layer-wise pipelining with separate CUDA streams:** One stream runs layer- l compute while another asynchronously loads KV for layer $l + 1$ into a fixed-size GPU buffer, so the model never waits for $a//KV$ before proceeding. Only a single-layer buffer is needed.
- **Asynchronous prefetch:** While a request is queued (admitted by scheduler but not yet running), LMCache prefetches its KV from slower tiers (remote/SSD) to faster ones (CPU or GPU), hiding I/O behind scheduling delays.
- **Process separation:** KV transfer runs in a separate process from the inference engine, avoiding 5–10% latency overhead from CPU contention and enabling a shared CPU memory pool across multiple engine instances.

(c) Minimize copies and duplicate data

- **Zero-copy with reference counting:** When the same KV is written to multiple destinations (CPU, disk, remote), LMCache keeps one physical copy and tracks references, instead of duplicating buffers.

- **Dynamic offloading of free pages:** For vLLM's free KV pages, it only offloads part of the free-page region rather than cloning everything, reducing both traffic and storage overhead. Net effect: KV transfers are (1) fewer and larger, (2) overlapped with inference, and (3) copy-minimal—so offloading/PD-disaggregation becomes practical instead of dominating latency.

2. Motivations & insights behind KV-cache sharing in DroidSpeak

Motivations

DroidSpeak targets **compound / multi-LLM systems** where multiple fine-tuned variants of the *same architecture* collaborate on the *same context*.

- **Multi-agent systems:** Different agents (coding, validation, debugging, etc.) talk to each other but all share the full conversation history as prefix.
- **Multi-LoRA / multi-fine-tuned models:** Many LoRA adapters or continuously updated fine-tunes over the same base model serve different tasks/users, often on the same document or conversation.
- **Personalized assistants:** Per-user fine-tuned models all read the same news/article/context but then respond differently.

In all these cases, **the expensive prefill over a long shared prefix is redundantly recomputed by each model**, even though they share architecture and context. Prior KV-prefix caching only works *within* one model; cross-model KV reuse could slash prefill time (e.g., 40K-token prefill from 4s → 0.08s if reuse were perfect). So the goal is: *reuse another model's KV where possible, without wrecking accuracy*.

Key empirical insights that shape DroidSpeak's design

DroidSpeak first runs a careful empirical study of cross-model KV reuse and derives three core insights:

1. **Insight 1 – Full KV reuse is too aggressive.**
Directly feeding the *entire* KV cache from a sender model into a different receiver model (same architecture, different weights) causes **huge accuracy loss**—e.g., HotpotQA can lose >50% F1 across many model pairs.
→ Motivation: we must do **selective** reuse, not all-layer reuse.
2. **Insight 2 – Only a small subset of layers are “critical”.**
When they reuse KV *one layer at a time* (recompute others), they find only ~11% of layers are highly sensitive (large F1 drop) and the rest can reuse sender KV with little quality loss; these sensitive ones are called **critical layers**.
→ Insight: **most layers' KV can be shared safely**; only a small critical subset needs recomputation.
3. **Insight 3 – Critical layers are stable across inputs.**
Layer-wise sensitivity patterns are similar across different examples: for non-critical layers, F1 change is tiny and low-variance; for critical layers, variance and magnitude are large.
→ Insight: **critical layers depend mainly on model pair, not on the specific input**, so you can profile once on a “training” dataset per model pair and then reuse that re-compute configuration in production.